

Actividad 1. Herramientas de Análisis con Meta AI.

Prompt 1. Explica la relación técnica entre NumPy y Pandas. ¿Por qué se dice que Pandas es una evolución de NumPy para el análisis de datos tabulares?.

Respuesta Meta:

NumPy y **Pandas** son las dos bibliotecas base para análisis de datos en Python, y Pandas está literalmente construido encima de NumPy. No compiten: Pandas es NumPy con esteroides para datos tabulares.

1. Relación técnica directa

NumPy es el motor de bajo nivel, Pandas es la carrocería

Aspecto	NumPy	Pandas
Estructura base	<code>ndarray</code> : array N-dimensional homogéneo. Todos los elementos son del mismo tipo	<code>Series</code> y <code>DataFrame</code> : estructuras 1D y 2D que usan <code>ndarray</code> por dentro
Tipos de datos	Homogéneo. Un array solo guarda <code>int64</code> , <code>float64</code> , etc	Heterogéneo por columnas. Cada columna de un <code>DataFrame</code> es un <code>ndarray</code> con su propio dtype
Índices	Solo enteros implícitos: 0, 1, 2...	Índices explícitos: fechas, strings, enteros. Puedes tener <code>df.loc['2026-01-01']</code>
Datos faltantes	No hay soporte nativo. Usas <code>np.nan</code> para float, pero es un hack	<code>pd.NA</code> , <code>np.nan</code> , <code>None</code> . Manejo nativo con <code>.isna()</code> , <code>.fillna()</code> , <code>.dropna()</code>
Operaciones	Matemáticas vectorizadas súper rápidas en C	Usa las operaciones de NumPy, pero añade groupby, merge, pivot, resample

Si abres el código fuente: `type(df['columna'].values)` te devuelve `numpy.ndarray` . Pandas es básicamente un wrapper inteligente alrededor de arrays de NumPy.

2. ¿Por qué Pandas es una "evolución" para datos tabulares?

NumPy nació en 2006 para hacer álgebra lineal y computación científica. Piensa en matrices, física, simulaciones. Es excelente para eso.

Pero datos tabulares del mundo real son un desastre:

1. **Columnas con tipos mezclados:** ID=int, nombre=string, fecha=datetime, ventas=float. En NumPy puro necesitarías `structured arrays` o `recarrays`, que son incómodos.
2. **Etiquetas importantes:** En finanzas quieres `df.loc['AAPL']`, no `array[347]`. NumPy no tiene índices con nombre.
3. **Datos faltantes por todos lados:** Encuestas, logs, CSVs rotos. NumPy te obliga a hacer malabares.
4. **Joins y merges:** Hacer un `SQL JOIN` con NumPy es escribir 50 líneas. En Pandas: `df1.merge(df2, on='id')`.
5. **Series de tiempo:** Resamplear de diario a mensual, manejar zonas horarias. NumPy no sabe qué es una fecha.

Pandas fue creado en 2008 por Wes McKinney en AQR Capital específicamente para resolver estos problemas en finanzas. Tomó NumPy y le añadió:

La capa tabular que faltaba:

- **Etiquetado automático:** Filas y columnas tienen nombres. `df['ventas']` en lugar de `arr[:, 3]`
- **Alineación automática:** `s1 + s2` alinea por índice, no por posición. Si falta un índice, pone `NaN`
- **Operaciones tipo SQL:** `groupby`, `merge`, `pivot_table`, `melt`. Todo lo que haces en Excel/DB
- **I/O robusto:** `read_csv`, `read_excel`, `read_sql`, `read_parquet` con inferencia de tipos y manejo de errores
- **Series de tiempo:** `resample`, `rolling`, `shift`, índices de fecha nativos

3. La analogía rápida

NumPy : Pandas :: C : Python

NumPy es rápido, de bajo nivel, sin opiniones. Tienes control total pero haces todo a mano.

Pandas es de alto nivel, opinionado, con batería incluida. Sacrificas un poco de velocidad pura por productividad brutal en tareas de datos tabulares.

Opinión: Se podría decir que pandas permite usar las herramientas de numpy (y otras nuevas) para manipular, visualizar y analizar dataframes, los cuales son variables particulares de la biblioteca pandas que facilitan el manejo de bases de datos. Pandas es fácil de entender y los dataframes permiten un manejo eficiente y sencillo de comprender para un usuario ajeno al desarrollo del código.

Prompt 2. Dame un ejemplo de la vida real (fuera de la programación) que ilustre la diferencia entre una Series y un DataFrame de Pandas.

Respuesta Meta:

Piensa en **Pandas** como tu forma de organizar información en tablas, como en Excel.

Una Series = Una sola columna con etiquetas

Ejemplo real: Tu lista del súper

Code

```
Manzanas:    6Leche:    2 litros  Pan:    1 piezaHuevos:    12Café:    1
bolsa
```

Aquí tienes una sola cosa que estás midiendo: "cantidad a comprar". Cada producto tiene su etiqueta y su valor. Eso es una Series.

Características de una Series:

1. **1 sola columna de datos:** Solo "cantidad"
2. **Índice:** Los nombres de los productos son las etiquetas
3. **1 tipo de información:** Todo son cantidades

Un DataFrame = Una tabla completa con varias columnas

Ejemplo real: Tu misma lista del súper, pero más detallada

Producto	Cantidad	Pasillo	Precio Unitario	¿Oferta?
Manzanas	6	3	\$8	No
Leche	2	6	\$25	Sí
Pan	1	4	\$35	No
Huevos	12	6	\$4	No
Café	1	5	\$120	Sí

Esto es un DataFrame.

Características de un DataFrame:

1. **Varias columnas:** Cantidad, Pasillo, Precio, Oferta
2. **Mismo índice para todas:** Los productos
3. **Cada columna es una Series:** La columna "Cantidad" sola es una Series. La columna "Precio" es otra Series. El DataFrame es básicamente un conjunto de Series que comparten el mismo índice.

La regla de oro para acordarte

Si en la vida real sería...	En Pandas es...
Una lista simple: "Calificaciones del grupo"	Series
Una tabla: "Calificaciones del grupo con nombre, materia, parcial y promedio"	DataFrame
El renglón "Temperatura" en el reporte del clima de la semana	Series
El reporte del clima completo con Temperatura, Humedad, Viento por día	DataFrame
Tu peso registrado cada mes del año	Series
Tu expediente médico con peso, presión, glucosa cada mes	DataFrame

Opinión: El objeto Dataframe de Pandas es una mezcla de diversos tipos de variables que permiten construir una tabla indexada que puede operarse como un documento de "excel" (guardando las distancias). Esto dota a estos objetos con cualidades que facilitan la operación de bases de datos.

Prompt 3. ¿En qué escenarios es superior utilizar Pandas en lugar de Microsoft Excel para el análisis de datos? Menciona al menos 3 ventajas técnicas relacionadas con el volumen de datos y la automatización.

¿Cuándo te conviene dejar Excel y pasarte a Pandas?

1. Cuando tus datos ya no caben en Excel

Excel tiene un límite duro: 1,048,576 filas por hoja. Si trabajas con logs de servidores, datos de sensores, transacciones de e-commerce o cualquier dataset de millones de registros, Excel simplemente se rinde o se congela.

Pandas: Maneja datasets de 10M, 50M+ de filas sin problema. La única limitante real es tu RAM. Y si ni así alcanza, puedes usar chunks o integrarlo con Dask/Spark para escalar a gigabytes/terabytes.

2. Cuando repites el mismo análisis cada semana/mes

En Excel, actualizar un reporte mensual implica: abrir el archivo, pegar datos nuevos, arrastrar fórmulas, actualizar tablas dinámicas, arreglar gráficos que se rompieron. Todo manual y propenso a errores.

Pandas: Escribes un script de 20 líneas una vez. Luego ejecutas `python reporte.py` y en 10 segundos tienes el CSV, Excel o dashboard actualizado. 100% reproducible y sin clicks.

3. Cuando necesitas análisis complejos o pipelines de datos

Si quieres hacer joins de 5 tablas, limpiar texto con regex, hacer groupby con funciones custom, series de tiempo con resampling, o modelos de ML, Excel se vuelve un laberinto de fórmulas anidadas y VBA.

Pandas: Todo eso es código legible. Un `df.merge().groupby().apply()` reemplaza 15 pasos manuales. Además se integra nativo con NumPy, Matplotlib, Scikit-learn, SQL, APIs, etc.

3 ventajas técnicas de Pandas vs Excel: Volumen y automatización

Ventaja	Excel	Pandas	Por qué importa
1. Escalabilidad de volumen	Máx 1,048,576 filas. Con 500k+ filas ya se alenta muchísimo.	Millones de filas limitado solo por RAM. Con <code>read_csv(chunksize=)</code> procesas archivos de cualquier tamaño.	Puedes analizar datos de producción reales sin muestrear ni dividir en 20 archivos.
2. Automatización y reproducibilidad	Macros VBA frágiles. Cualquier cambio manual rompe la trazabilidad. Difícil versionar.	Scripts <code>.py</code> que se ejecutan idéntico siempre. Se versiona con Git, se programa con cron/Airflow, se integra a CI/CD.	Cero error humano. Un analista nuevo corre tu script y obtiene el mismo resultado. Ahorras horas cada ciclo.
3. Rendimiento en operaciones vectorizadas	Fórmulas se recalculan celda por celda. Una <code>BUSCARV</code> en 1M filas tarda minutos.	Operaciones vectorizadas en C. <code>df['col'] * 2</code> procesa 10M filas en <1 segundo. Joins y groupby optimizados.	Lo que en Excel toma 15 min y cuelga el programa, en Pandas son 3 segundos.

Opinión: Excel resulta ser de gran utilidad cuando se manejan bases de datos chicas (~1000000 datos), aunque incluso con estas dimensiones puede resultar pesado para el software pues debe cargar todos los datos cada vez que se abre, guarda y escribe sobre el documento. Excel tiende a saturarse con tantos datos debido a su manejo de la memoria del sistema por lo que con grandes bases de datos se mantiene inferior frente a programas/librerías de manejos de datos como SQL, Python, Julia, etc.